

MID-PROJECT PRESENTATION

ClairSec

An adversarial multi-agent platform that finds, fixes and verifies REST APIs

Builder

Attacker

Evaluator

Fixer

Verifier

Zaid Sayyed

Chirag Kadam

The problem we set out to solve

Most web applications today are collections of REST APIs. The security burden has moved to that API layer — and the most common flaw there is not a coding mistake a tool can pattern-match.

- **Broken authorization is #1.** The top API risk is code that checks who you are, but forgets to check what you're allowed to touch.
- **It looks completely normal.** The request is valid, authenticated, passes a firewall, returns 200. Only the meaning is wrong.
- **So scanners miss it.** There is no signature for "this user shouldn't own this record" — it depends on the app's own data model.

WHAT A VULNERABLE ENDPOINT LOOKS LIKE

```
@app.get("/documents/{doc_id}")
def read(doc_id, user = Depends(auth)):
    doc = DOCUMENTS.get(doc_id)
    # authenticated, but never checks
    # whether this user owns the doc
    return doc
```

Any logged-in user can read anyone's document.

Why the existing options fall short

Traditional scanners

Reliable and repeatable

But cannot reason about your app's ownership rules

So authorization flaws slip through

A single AI agent

Can read and reason about code

But reports things that aren't really there

And confirms its own guesses

What we wanted

Reason like the AI approach

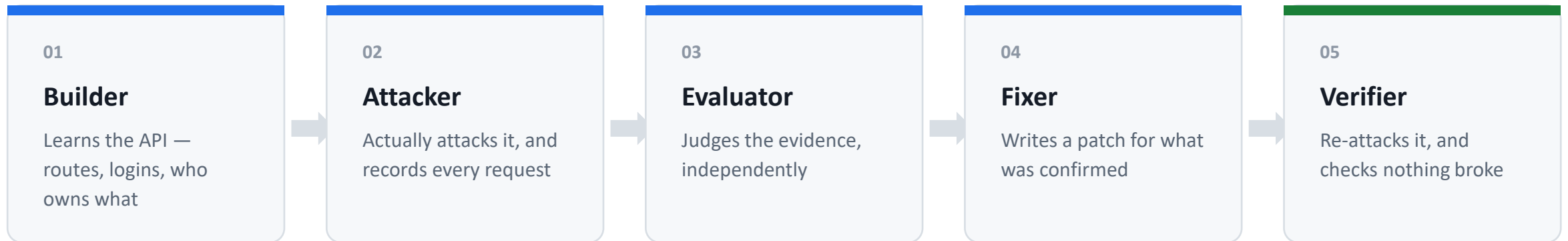
Prove it like a real attacker would

Judge it with something independent

Our approach: never report a vulnerability the system hasn't actually exploited — and never let the agent that found it be the one that confirms it.

What ClairSec does

You point it at a FastAPI project. It runs the app in a sealed container and puts five specialists to work on it.



Throughout all of this, your original code is never touched. Every change happens inside an isolated copy.

The decision everything rests on

An AI never decides whether a security test passed. Ordinary code does, by looking at what actually came back over HTTP.

IF THE MODEL JUDGED ITSELF

We would only be measuring whether the AI agrees with itself. It could confidently report a vulnerability that was never really there — and nothing in the system could catch it.

HOW WE ACTUALLY DO IT

User A asks for User B's document. If the reply is 200 and contains B's data, that's a real flaw. That check is plain code — same input, same answer, every time.

The AI is used where it's genuinely good — understanding code and writing fixes. It is kept away from deciding what counts as proof.

How it's built

Desktop app

Flutter · Dart · Riverpod

What the user sees and clicks

Backend

Python · FastAPI · MongoDB

Runs the agents, owns all the logic

Isolation

Docker · internal network

Where the target app actually runs

SAFETY RULES

Target runs with no network access out

No root, no writable filesystem

The AI can propose, never execute

Your source is never modified

How we know it actually works

We built two versions of the same small API — so we can measure both what it catches and what it wrongly flags.

The vulnerable version

3 flaws planted on purpose, recorded before any scan

Broken authorization · mass assignment · misconfiguration

Every flaw has a script that proves it is exploitable

The secure version

Same endpoints, same behaviour — but written correctly

Anything reported here is a false alarm, by definition

Without this, a tool could 'pass' by flagging everything

We wrote both applications ourselves so the AI could not have seen them, or their fixes, anywhere before.

FINDINGS

Results so far

3 / 3

Planted flaws found
on the vulnerable app

0

False alarms
on the secure app

3 / 3

Fixes verified
attack blocked, app still works

~15s

Full scan
after the first build

WHAT IT FOUND

Broken object authorization

GET /documents/{doc_id}

CWE-639

Fix verified

Mass assignment

PUT /users/{user_id}/profile

CWE-915

Fix verified

Security misconfiguration

GET /debug/config

CWE-16

Fix verified

"Fixed" has to mean something

Any tool can stop an attack by deleting the feature it targets. So we refuse to call a fix successful unless two things are true at once.



The attack no longer works

We rebuild the app with the patch and run the exact same attack again.



Nothing else broke

We run the app's own test suite. All 6 tests still pass — the feature still works for legitimate users.

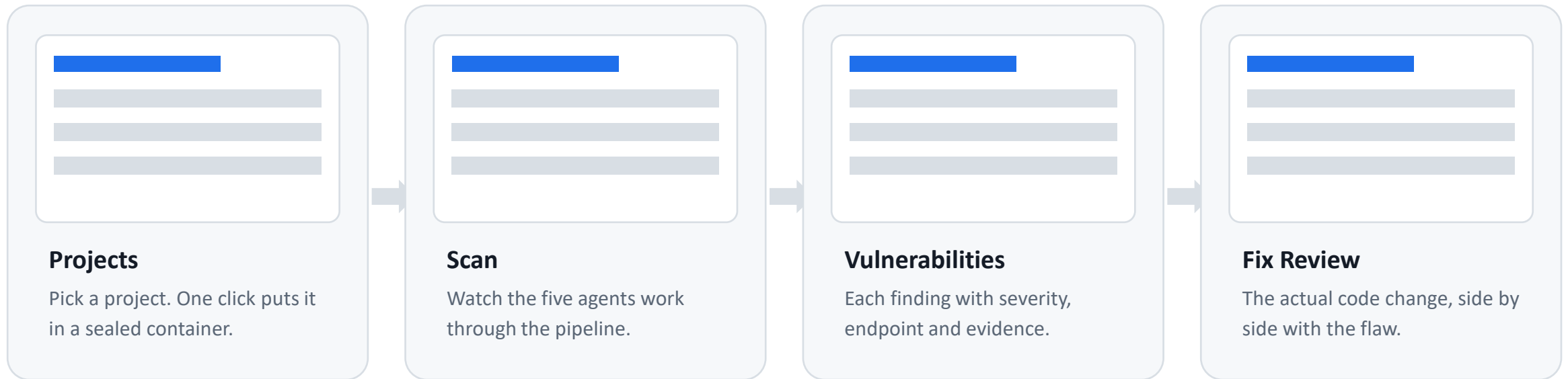


It holds up to a variation

We try a modified version of the attack, to check we fixed the real weakness and not just one route to it.

All three checks passed on all three fixes.

What it looks like to use



Replace this slide with your own screenshots before presenting — the Fix Review screen showing a real diff is the strongest one to show.

PROGRESS

Where we are

COMPLETED

- | | |
|-----------|---|
| Phase 0–2 | Setup, desktop app, project import |
| Phase 3 | Docker isolation and container lifecycle |
| Phase 4 | Builder agent and the whole AI layer |
| Phase 5 | Attacker agent and the deterministic checks |
| Phase 6 | Evaluator agent, severity scoring |
| Phase 7 | Fixer agent and patch safety |
| Phase 8 | Re-testing and verification |

STILL TO COME

- | | |
|----------|-----------------------------------|
| Phase 9 | Live progress streaming in the UI |
| Phase 10 | Exportable reports |
| Phase 11 | Full comparison study |

BUILT SO FAR

- 171 automated tests, all passing
- 5 agents working end to end
- 16 specification documents

What we ran into

Isolation was harder than expected

Sealing the container off from the network also blocked our own health checks. We had to add a small proxy that sits on both sides.

A convincing fix isn't always a good fix

Early on we only checked that the attack stopped working. Then we realised deleting the endpoint would 'pass' that test too.

The AI had to be kept on a short leash

We stopped letting it produce file paths or commands directly. It proposes; our code decides whether the proposal is allowed.

Testing broke our own app

Our test suite was overwriting the running app's login token. Small bug, but it cost us a lot of confusing debugging.

What's next

1

Live progress in the UI

Watch each agent work in real time instead of waiting for the result.

2

Exportable reports

A document a developer can act on without opening our app.

3

More vulnerability types

We deliberately started with three we can prove. Next: server-side request forgery and rate-limit abuse.

4

The comparison study

Measure our approach against a traditional scanner and a single-agent setup, on the same projects, with proper statistics.

Thank you

A security tool is only useful if you can trust what it tells you. That is what we set out to build.

Zaid Sayyed
Chirag Kadam

Questions welcome